Experiment No.05

Aim: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

Theory:

1. Dataset Source

Dataset: Pima Indians Diabetes

Source: Kaggle

Link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

2. Dataset Description

The Pima Indians Diabetes Dataset contains medical data collected from female patients of Pima Indian heritage aged at least 21 years. The dataset is used to predict whether a patient is diabetic.

```
df = pd.read_csv("sample_data/diabetes.csv")

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Target Variable

- 0 → Non-diabetic
- 1 → Diabetic

This dataset is commonly used to build models for medical diagnosis classification problems.

3. Mathematical Formulation of the Algorithm

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. The objective of SVM is to find the optimal hyperplane that separates different classes with maximum margin.

Hyperplane Equation:

$$w \cdot x + b = 0$$

The classification decision rule is:

$$y = sign(w \cdot x + b)$$

Margin Maximization

SVM maximizes the distance between the hyperplane and the nearest data points (support vectors).

Optimization problem:

$$\min \frac{1}{2}||w||^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

Kernel Trick

For non-linear data, SVM uses kernel functions such as:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF)
- Sigmoid Kernel

In this experiment, the RBF kernel was used.

4. Algorithm Limitations

Although SVM is a powerful classification algorithm, it has several limitations:

- Computationally expensive for large datasets
- Sensitive to choice of kernel and hyperparameters
- Requires feature scaling
- Not suitable when the dataset contains many noisy observations
- Difficult to interpret compared to simpler models
- Performance decreases when classes overlap significantly

5. Methodology / Workflow

The following steps were followed to perform the experiment.

Step 1: Dataset Collection

The Pima Indians Diabetes dataset was downloaded from Kaggle.

Step 2: Data Loading

The dataset was loaded into Python using the Pandas library.

Step 3: Data Preprocessing

- Checked dataset structure using df.head()
- Verified dataset statistics using df.describe()
- Identified input features and target variable

Step 4: Feature Scaling

Since SVM is distance-based, StandardScaler was applied to normalize feature values.

Step 5: Train-Test Split

The dataset was divided into:

- Training Data (80%)
- Testing Data (20%)

Step 6: Model Training

A Support Vector Machine classifier was trained using the training dataset.

Step 7: Model Prediction

The trained model predicted whether patients in the test dataset were diabetic or non-diabetic.

Step 8: Model Evaluation

Model performance was evaluated using:

- Accuracy Score
- Confusion Matrix

Step 9: Hyperparameter Optimization

GridSearchCV was used to find the optimal values of C and gamma.

6. Performance Analysis

Model performance was evaluated using accuracy score and confusion matrix.

Accuracy Formula

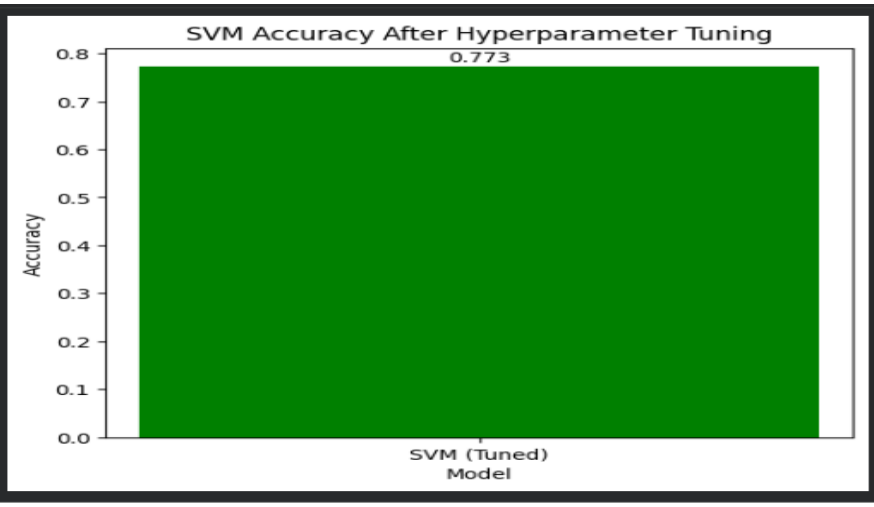
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

```python
accuracy = accuracy_score(y_test, y_pred)

accuracy_df = pd.DataFrame({
    "Model": ["Support Vector Machine"],
    "Accuracy": [accuracy]
})

accuracy_df
```

	Model	Accuracy
0	Support Vector Machine	0.733766



7. Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV.

Parameters Tuned

1. C (Regularization Parameter)
   Controls the trade-off between margin width and classification error.

2. Gamma
   Controls the influence of individual training samples.

Parameter Grid

C = [0.1, 1, 10, 100]

gamma = [1, 0.1, 0.01, 0.001]

kernel = ['rbf']

Process

- Multiple parameter combinations were tested.
- Cross-validation was performed to evaluate performance.
- The best parameter combination was selected based on highest accuracy.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# -------------------------------
# STEP 1: Load dataset
# -------------------------------
df = pd.read_csv("sample_data/diabetes.csv")

print("Total records:", len(df))
print("Columns:", df.columns)

# -------------------------------
# STEP 2: Features & Target
# -------------------------------
# Use 2 features for visualization (important for decision boundary)
```

```
X = df[["Glucose", "BMI"]]
y = df["Outcome"]

# -------------------------------
# STEP 3: Scaling
# -------------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# -------------------------------
# STEP 4: Train-test split
# -------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# -------------------------------
# STEP 5: Train SVM
# -------------------------------
model = SVC(kernel="linear", class_weight="balanced")
model.fit(X_train, y_train)

# -------------------------------
# STEP 6: Prediction
# -------------------------------
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

# -------------------------------
# STEP 7: Decision Boundary
# -------------------------------
plt.figure(figsize=(8,6))

plt.scatter(
    X_train[:,0], X_train[:,1],
    c=y_train,
    edgecolors='k',
    alpha=0.7
)

plt.xlabel("Glucose (scaled)")
plt.ylabel("BMI (scaled)")
plt.title("SVM Classification (Diabetes Dataset)")

# Decision boundary
w = model.coef_[0]
```

```
b = model.intercept_[0]

x_vals = np.linspace(X_train[:,0].min(), X_train[:,0].max(), 100)
y_vals = -(w[0]*x_vals + b) / w[1]

plt.plot(x_vals, y_vals)
plt.show()

# -------------------------------
# STEP 8: Confusion Matrix
# -------------------------------
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Diabetes", "Diabetes"]
)

# Change color here
disp.plot(cmap="Blues")   # try: Reds, Greens, Oranges, Purples, coolwarm

plt.title("Confusion Matrix")
plt.show()
```

Output:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
60	2	84	0	0	0	0.0	0.304	21
618	9	112	82	24	0	28.2	1.282	50
346	1	139	46	19	83	28.7	0.654	22
294	0	161	50	0	0	21.9	0.254	65
231	6	134	80	37	370	46.2	0.238	46

Next steps: Generate code with X_train New interactive sheet

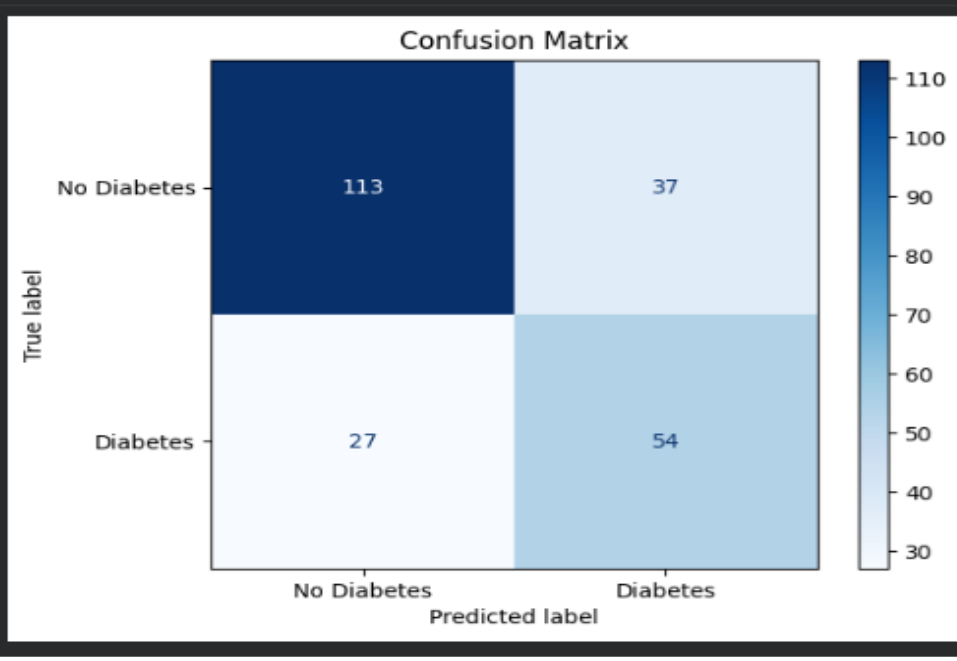
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

pd.DataFrame(X_train).head()
```

	0	1	2	3	4	5	6	7
0	-0.526397	-1.151398	-3.752683	-1.322774	-0.701206	-4.135256	-0.490735	-1.035940
1	1.588046	-0.276643	0.680345	0.233505	-0.701206	-0.489169	2.415030	1.487101
2	-0.828460	0.566871	-1.265862	-0.090720	0.013448	-0.424522	0.549161	-0.948939
3	-1.130523	1.254179	-1.049617	-1.322774	-0.701206	-1.303720	-0.639291	2.792122
4	0.681856	0.410665	0.572222	1.076490	2.484601	1.838121	-0.686829	1.139095

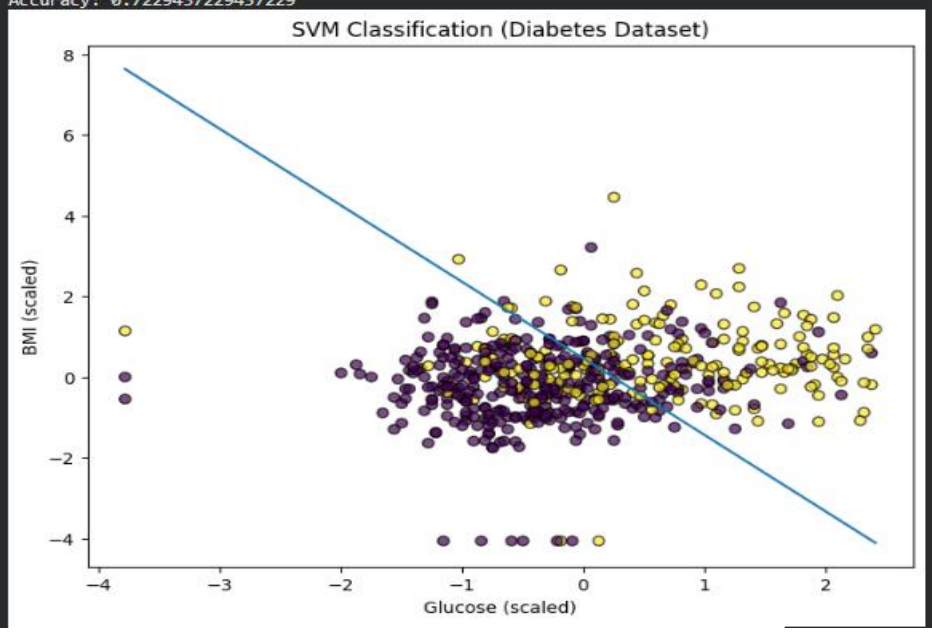
```python
svm_model = SVC(kernel="rbf")

svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)

pd.DataFrame(y_pred, columns=["Predicted Outcome"]).head()
```

	Predicted Outcome
0	0
1	0
2	0
3	0
4	0

```python
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Diabetes", "Diabetes"]
)

# Change color here
disp.plot(cmap="Blues")   # try: Reds, Greens, Oranges, Purples, coolwarm

plt.title("Confusion Matrix")
plt.show()
```



```
Total records: 768
Columns: Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
Accuracy: 0.7229437229437229
```




```
param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": [1, 0.1, 0.01, 0.001],
    "kernel": ["rbf"]
}

grid = GridSearchCV(
    SVC(),
    param_grid,
    refit=True,
    cv=5
)

grid.fit(X_train, y_train)

best_params_df = pd.DataFrame([grid.best_params_])

best_params_df
```

	C	gamma	kernel
0	100	0.001	rbf

```
best_model = grid.best_estimator_

y_pred_best = best_model.predict(X_test)

best_accuracy = accuracy_score(y_test, y_pred_best)

tuned_accuracy_df = pd.DataFrame({
    "Model": ["SVM (Tuned)"],
    "Accuracy": [best_accuracy]
})

tuned_accuracy_df
```

	Model	Accuracy
0	SVM (Tuned)	0.772727

Conclusion:

In this experiment, Support Vector Machine (SVM) was implemented to classify diabetes cases using the Pima Indians Diabetes dataset. The model successfully predicted diabetic outcomes based on medical attributes. Hyperparameter tuning using GridSearchCV improved the model's accuracy and performance. The experiment demonstrates that SVM is an effective algorithm for classification problems involving medical datasets.